

Documentación Técnica

# Plataforma de Feedback

Aram Sana

21 de marzo de 2026

## Índice

---

|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>1. Descripción General</b>                                   | <b>2</b> |
| <b>2. Requisitos Previos</b>                                    | <b>2</b> |
| <b>3. Instalación y Configuración</b>                           | <b>2</b> |
| 3.1. Clonar el repositorio . . . . .                            | 2        |
| 3.2. Crear el entorno virtual e instalar dependencias . . . . . | 2        |
| 3.3. Configurar las credenciales de Google . . . . .            | 2        |
| 3.4. Configurar Apps Script . . . . .                           | 2        |
| <b>4. Estructura del Proyecto</b>                               | <b>3</b> |
| <b>5. Ejecución</b>                                             | <b>3</b> |
| <b>6. Flujo de Datos</b>                                        | <b>3</b> |
| <b>7. Sanitización de Entradas</b>                              | <b>3</b> |
| <b>8. Recursos</b>                                              | <b>4</b> |

## Descripción General

---

Este proyecto implementa una plataforma centralizada de recolección y análisis de retroalimentación para productos de Alegra. El sistema permite a los usuarios enviar comentarios a través de una interfaz web, los cuales se almacenan automáticamente en una hoja de cálculo de Google Sheets. Cada registro es procesado por funciones de Apps Script que utilizan la API de Gemini y el modelo de Gemma para clasificar el sentimiento del comentario y generar un resumen automatizado. Los resultados se visualizan en un dashboard interactivo de Looker Studio.

El sistema está compuesto por tres capas: una aplicación web en Flask que gestiona la captura del feedback, una hoja de cálculo de Google Sheets como base de datos, y un conjunto de funciones de Apps Script que ejecutan el análisis con inteligencia artificial.

## Requisitos Previos

---

Para ejecutar el proyecto es necesario contar con Python 3.10 o superior, una cuenta de Google con acceso a Google Sheets y Google Cloud, y una clave de API de Gemini. Las dependencias de Python se instalan a través del archivo `requirements.txt` incluido en el repositorio.

## Instalación y Configuración

---

### Clonar el repositorio

```
1 git clone https://github.com/AramJSana/alegra-feedback.git
2 cd alegra-feedback
```

### Crear el entorno virtual e instalar dependencias

```
1 python -m venv .venv
2 source .venv/Scripts/activate
3 pip install -r requirements.txt
```

### Configurar las credenciales de Google

Es necesario crear un proyecto en Google Cloud Console, habilitar la API de Google Sheets, y generar una cuenta de servicio con permiso de editor. El archivo JSON de credenciales debe colocarse en la carpeta `KEYS/` en la raíz del proyecto con el nombre exacto que se referencia en `app.py` o en su defecto modificar el código para cambiar el nombre del archivo.

Una vez creada la cuenta de servicio, se le comparte acceso a la hoja de cálculo con permisos de editor.

### Configurar Apps Script

Los archivos `sentiment.gs` y `summary.gs` deben copiarse en el editor de Apps Script de la hoja de cálculo. La clave de API de Gemini se configura como una propiedad de script teniendo como objetivo evitar exponerla en el código fuente. En el editor de

Apps Script, navegar a Proyecto, Propiedades del script, y agregar una propiedad llamada GEMINI\_API\_KEY con el valor de la clave obtenida desde Google AI Studio.

## Estructura del Proyecto

| Archivo                | Descripción                                              |
|------------------------|----------------------------------------------------------|
| app.py                 | Aplicación principal en Flask                            |
| requirements.txt       | Dependencias de Python                                   |
| sentiment.gs           | Función de Apps Script para clasificación de sentimiento |
| summary.gs             | Función de Apps Script para generación de resumen        |
| templates/base.html    | Plantilla base HTML                                      |
| templates/index.html   | Formulario de captura de feedback                        |
| templates/success.html | Página de confirmación de envío                          |
| templates/error.html   | Página de error                                          |
| static/styles.css      | Estilos de la interfaz                                   |
| KEYS/googlekey.json    | Credenciales de la cuenta de servicio                    |

## Ejecución

Con el entorno virtual activo, ejecutar el servidor de desarrollo con el siguiente comando:

```
1 flask run
```

La aplicación estará disponible en la IP local por defecto `http://127.0.0.1:5000`.

## Flujo de Datos

El usuario completa el formulario con su nombre (opcional), el producto que desea evaluar y su comentario. Al enviar el formulario, la aplicación Flask valida los datos, sanitiza las entradas para prevenir inyección de fórmulas en la hoja de cálculo, y agrega una nueva fila en Google Sheets con la marca de tiempo, producto, comentario, nombre del usuario, y las fórmulas `=sentiment()` y `=summary()` en las columnas correspondientes. La marca de tiempo sigue el formato que se encontraba en la base de datos inicial, provisionada por Alegra.

Cuando Apps Script evalúa estas fórmulas, consulta la API de Gemini con el contenido del comentario. La función `sentiment()` retorna un valor numérico de 0 (negativo), 1 (neutral) o 2 (positivo) o un valor textual de “NULL” en caso de que el comentario sea spam o no sea inteligible. La función `summary()` retorna un resumen breve en español generado por el modelo. `summary()` también revisa si el comentario es spam y en caso de que sea así retorna “Opinión inválida”; esto con el objetivo de limpiar datos basura.

## Sanitización de Entradas

Para prevenir que un usuario inyecte fórmulas en la hoja de cálculo mediante los campos de texto, la aplicación aplica la siguiente función de sanitización antes de escribir cualquier valor proporcionado por el usuario:

```
1 def sanitize(value):  
2     if value and value[0] in ("=", "+", "-", "@"):  
3         value = "'" + value  
4     return value
```

Cualquier valor que comience con un carácter que Google Sheets interpretaría como fórmula recibe un apóstrofo como prefijo, lo que fuerza a la celda a tratarlo como texto.

## Recursos

---

**Repositorio** Código fuente del proyecto en GitHub.

[Abrir en Github](#)

**Hoja de cálculo** Base de datos en Google Sheets.

[Abrir en Google Sheets](#)

**Dashboard** Dashboard en Looker Studio.

[Abrir en Looker Studio](#)